Python Data Analytics Workshop

Introduction

This workshop is designed to reinforce the key concepts covered in the Python Data Analytics Bootcamp. We'll go through a series of exercises covering:

- Conditional Statements
- Functions
- Lists
- Dictionaries
- Loops

Let's get started!

In [2]:
```
## Part 1: Conditional Statements
### Basic if statement
# Let's check if a number is positive, negative, or zero
number = 5  # Try changing this value

if number > 0:
    print(f"{number} is positive")
elif number < 0:
    print(f"{number} is negative")
else:
    print(f"{number} is zero")
```

```
5 is positive
```

In [16]:
```
#my answer

number = 5
if number > 0:
    print(f"{number}  positive")
if number <0:
    print(f"{number}  negative")
if number ==0:
    print (f"{number} zero")
```

```
5  positive
```

In [12]:
```
### Exercise 2: Discount Calculator
# Write a function that calculates the final price after applying a discount
# If the purchase amount is $100 or more, apply a 10% discount
# If the purchase amount is $500 or more, apply a 20% discount
# If the purchase amount is $1000 or more, apply a 30% discount

def calculate_discount(purchase_amount):
    # Your code here
    if purchase_amount >= 1000:
```

```
        discount = 0.3
    elif purchase_amount >= 500:
        discount = 0.2
    elif purchase_amount >= 100:
        discount = 0.1
    else:
        discount = 0

    final_price = purchase_amount - (purchase_amount * discount)
    return final_price

# Test your function
purchase_amounts = [50, 150, 600, 1200]
for amount in purchase_amounts:
    print(f"Original price: ${amount}, Final price: ${calculate_discount(amount):.2
```

```
Original price: $50, Final price: $50.00
Original price: $150, Final price: $135.00
Original price: $600, Final price: $480.00
Original price: $1200, Final price: $840.00
```

In [13]:
```
#my answer

def calculate_discount (purchase_amount):
    if purchase_amount >= 1000:
        discount = 0.3
    elif purchase_amount >= 500:
        discount = 0.2
    elif purchase_amount >= 100:
        discount = .1
    else:
        discount = 0

    final_price = purchase_amount - (purchase_amount * discount)
    return final_price

#test your function

Purchase_amounts = [50, 150, 600, 1200]
for amount in purchase_amounts:
    print (f"Original price: ${amount}, Final price: ${calculate_discount(amount):.
```

```
Original price: $50, Final price: $50.00
Original price: $150, Final price: $135.00
Original price: $600, Final price: $480.00
Original price: $1200, Final price: $840.00
```

In [10]:
```
#my answer (I deleted the original question)

def calculate_grade(score):
    if 90 <= score <=100:
        return "A"
    elif 80 <= score <= 90:
        return "B"
    elif 70 <= score <= 80:
        return "C"
    elif 60 <= score <= 70:
```

```
        return "D"
    else:
        return "F"

#test function
test_scores = [95, 82, 71, 65, 55]
for score in test_scores:
    print(f"Score: {score}, Grade: {calculate_grade(score)}")
```

```
Score: 95, Grade: A
Score: 82, Grade: B
Score: 71, Grade: C
Score: 65, Grade: D
Score: 55, Grade: F
```

In [20]:
```
### Exercise 3: Temperature Converter
# Write a function that converts temperature from Celsius to Fahrenheit
# Formula: F = C * 9/5 + 32

def celsius_to_fahrenheit(celsius):
    # Your code here
    fahrenheit = celsius * 9/5 + 32
    return fahrenheit

# Test your function
temperatures_c = [0, 25, 37, 100]
for temp in temperatures_c:
    print(f"{temp}°C = {celsius_to_fahrenheit(temp):.1f}°F")
```

```
0°C = 32.0°F
25°C = 77.0°F
37°C = 98.6°F
100°C = 212.0°F
```

In [21]:
```
#my amswer

def celsius_to_farrenheit(celsius):
    fahrenheit = celsius * 9/5 + 32
    return fahreheit

temperatures_c = [0, 25, 37, 100]
for temp in temperatures_c:
    print(f"{temp}°C = {celsius_to_fahrenheit(temp):.1f}°F")
```

```
0°C = 32.0°F
25°C = 77.0°F
37°C = 98.6°F
100°C = 212.0°F
```

In [5]:
```
## Part 2: Functions
### Basic function definition
def greet(name):
    """A simple function that greets a person by name"""
    return f"Hello, {name}!"

# Call the function
print(greet("Data Analyst"))
```

```
Hello, Data Analyst!
```

In [6]:
```python
#my answer

def greet (name):
    return f"Hello, {name}!"

#call the function
print (greet ("Mr. Paul, Data Analyst"))
```

```
Hello, Mr. Paul, Data Analyst!
```

In [10]:
```python
### Exercise 4: Calculator Function
# Write a function that takes two numbers and an operation as parameters
# Supported operations: add, subtract, multiply, divide
# Return the result of the operation

def calculator(num1, num2, operation):
    # Your code here
    if operation == "add":
        return num1 + num2
    elif operation == "subtract":
        return num1 - num2
    elif operation == "multiply":
        return num1 * num2
    elif operation == "divide":
        if num2 == 0:
            return "Error: Division by zero"
        return num1 / num2
    else:
        return "Error: Unknown operation"

# Test your function
print(calculator(10, 5, "add"))      # Should return 15
print(calculator(10, 5, "subtract")) # Should return 5
print(calculator(10, 5, "multiply")) # Should return 50
print(calculator(10, 5, "divide"))   # Should return 2
print(calculator(10, 0, "divide"))   # Should return an error
```

```
15
5
50
2.0
Error: Division by zero
```

In [15]:
```python
#my answer

def calculator(num1, num2, operation):
    if operation == "add":
        return num1 + num2
    elif operation == "subtract":
        return num1 - num2
    elif operation == "multiply":
        return num1 * num2
    elif operation == "divide":
        if num2 == 0:
            return "Error: Division by zero"
```

```
        return num1 / num2
    else:
        return "error: unknown operation"

print (calculator (10, 5, "add"))
print (calculator (10, 5, "subtract"))
print (calculator (10, 5, "multiply"))
print (calculator (10, 5, "divide"))
print (calculator (10, 0, "divide"))
```

```
15
5
50
2.0
Error: Division by zero
```

In [16]:
```
## Part 3: Lists

### Basic list operations
fruits = ["apple", "banana", "cherry", "date"]
print("Original list:", fruits)

# Accessing elements
print("First fruit:", fruits[0])
print("Last fruit:", fruits[-1])

# Modifying the list
fruits[1] = "blueberry"
print("After modification:", fruits)

# Adding elements
fruits.append("elderberry")
print("After append:", fruits)

# Removing elements
removed_fruit = fruits.pop(0)
print(f"Removed {removed_fruit}, List now:", fruits)
```

```
Original list: ['apple', 'banana', 'cherry', 'date']
First fruit: apple
Last fruit: date
After modification: ['apple', 'blueberry', 'cherry', 'date']
After append: ['apple', 'blueberry', 'cherry', 'date', 'elderberry']
Removed apple, List now: ['blueberry', 'cherry', 'date', 'elderberry']
```

In [11]:
```
#my answer

fruits = ["pears", "watermelon", "orange", "grape"]
print("Original list:", fruits)

print("First fruits:", fruits[0])
print("Last fruits:", fruits[-1])

#modify the list
fruits[1] = "approcote"
print("After modification:", fruits)
```

```
#adding eLements
fruits.append("peach")
print("After append:", fruits)

#removing elements
removed_fruit = fruits.pop(0)
print(f"Removed {removed_fruit}, List now:", fruits)
```

```
Original list: ['pears', 'watermelon', 'orange', 'grape']
First fruits: pears
Last fruits: grape
After modification: ['pears', 'approcote', 'orange', 'grape']
After append: ['pears', 'approcote', 'orange', 'grape', 'peach']
Removed pears, List now: ['approcote', 'orange', 'grape', 'peach']
```

In [ ]:
```
### Exercise 5: List Statistics
# Write a function that takes a list of numbers and returns a dictionary with:
# - The minimum value
# - The maximum value
# - The average value
# - The sum of all values

def calculate_statistics(numbers):
    # Your code here
    if not numbers:
        return {
            "min": None,
            "max": None,
            "average": None,
            "sum": None
        }

    return {
        "min": min(numbers),
        "max": max(numbers),
        "average": sum(numbers) / len(numbers),
        "sum": sum(numbers)
    }

# Test your function
test_numbers = [5, 12, 8, 42, 23, 16]
stats = calculate_statistics(test_numbers)
print("Statistics:", stats)
```

In [1]:
```
#my answer

def calculate_statistics (numbers):
    return {
        "minimum value": min(numbers),
        "maximum value": max(numbers),
        "average value": sum(numbers)/len(numbers),
        "sum": sum (numbers)
    }

test_numbers = [5,12,8,42,23,16]
```

```
stats = calculate_statistics(test_numbers)
print("statistics:",stats)
```

```
statistics: {'minimum value': 5, 'maximum value': 42, 'average value': 17.6666666666
66668, 'sum': 106}
```

In [ ]:
```
### Exercise 6: List Filtering
# Write a function that takes a list of numbers and returns a new list containing o

def filter_even_numbers(numbers):
    # Your code here
    return [num for num in numbers if num % 2 == 0]

# Test your function
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_numbers = filter_even_numbers(numbers)
print("Even numbers:", even_numbers)
```

In [ ]:
```
## Part 4: Dictionaries
### Basic dictionary operations
student = {
    "name": "Alex",
    "age": 23,
    "major": "Data Science",
    "gpa": 3.8
}

print("Student info:", student)

# Accessing values
print("Name:", student["name"])
print("Major:", student.get("major"))

# Adding or modifying key-value pairs
student["year"] = "Senior"
student["gpa"] = 3.9
print("Updated student info:", student)

# Removing key-value pairs
removed_age = student.pop("age")
print(f"Removed age: {removed_age}, Student info now:", student)
```

In [16]:
```
#my answer

student = {
    "name": "Bart",
    "age": 10,
    "major": "General",
    "gpa": 0.0
}

print ("student info:", student)

#Accessing values
print("Name:", student["name"])
print("Major", student.get("major"))
```

```
#adding or modifying key-value pairs
student ["year"] = "Freshman"
student["gpa"] = 0.0
print("Updated Student Info", student)

#remove key-value pairs
removed_age = student.pop("age")
print(f"Removed age: {removed_age}, Student Info Now:", student)
```

```
student info: {'name': 'Bart', 'age': 10, 'major': 'General', 'gpa': 0.0}
Name: Bart
Major General
Updated Student Info {'name': 'Bart', 'age': 10, 'major': 'General', 'gpa': 0.0, 'ye
ar': 'Freshman'}
Removed age: 10, Student Info Now: {'name': 'Bart', 'major': 'General', 'gpa': 0.0,
'year': 'Freshman'}
```

In [33]:
```
### Exercise 7: Word Counter
# Write a function that takes a string and returns a dictionary with each word as a
# and the number of occurrences as the value

def count_words(text):
    # Your code here
    words = text.lower().split()
    word_count = {}

    for word in words:
        # Remove punctuation if needed
        word = word.strip(".,!?;:\"'()")
        if word in word_count:
            word_count[word] += 1
        else:
            word_count[word] = 1

        return word_count

# Test your function
sample_text = "The quick brown fox jumps over the lazy dog. The fox was quick and t
word_counts = count_words(sample_text)
print("Word counts:", word_counts)
```

```
Word counts: {'the': 1}
```

In [32]:
```
#my answer

def count_words(text):
    words = text.lower().split()
    word_count = {}

    word = word.strip(".,!?:;\"'()")
    if word in word_count:
        word_count[word] += 1
    else:
        word_count[word] = 1

    return word_count
```

```
sample_text = "The quick fox jumps over the lazy dog.  The fox was quick and the do
word_counts = count_words(sample_text)
print("Word counts:", word_counts)
```

```
---------------------------------------------------------------------------
UnboundLocalError                         Traceback (most recent call last)
Cell In[32], line 14
     11     return word_count
     13 sample_text = "The quick fox jumps over the lazy dog.  The fox was quick and
the dog was lazy."
---> 14 word_counts = count_words(sample_text)
     15 print("Word counts:", word_counts)

Cell In[32], line 5, in count_words(text)
      2 words = text.lower().split()
      3 word_count ={}
----> 5 word = word.strip(".,!?:;\"'()")
      6 if word in word_count:
      7     word_count[word] += 1

UnboundLocalError: local variable 'word' referenced before assignment
```

In [ ]:
```
### Exercise 8: Contact Manager
# Create a small contact manager that lets you add, update, and retrieve contact in

contact_book = {}

def add_contact(name, phone, email=None):
    contact_book[name] = {"phone": phone, "email": email}
    return f"Added contact: {name}"

def update_contact(name, phone=None, email=None):
    if name not in contact_book:
        return f"Contact {name} not found"

    if phone:
        contact_book[name]["phone"] = phone
    if email:
        contact_book[name]["email"] = email

    return f"Updated contact: {name}"

def get_contact(name):
    return contact_book.get(name, f"Contact {name} not found")

# Test your contact manager
print(add_contact("Alice", "555-1234", "alice@example.com"))
print(add_contact("Bob", "555-5678"))
print(get_contact("Alice"))
print(update_contact("Alice", email="alice.new@example.com"))
print(get_contact("Alice"))
print(get_contact("Charlie"))
```

In [ ]:
```
## Part 5: Loops
### For loop with a list
```

```
print("Iterating through a list:")
colors = ["red", "green", "blue", "yellow"]
for color in colors:
    print(f"Color: {color}")

### For loop with range
print("\nUsing range():")
for i in range(5):
    print(f"Number: {i}")

### While loop
print("\nUsing a while loop:")
count = 0
while count < 5:
    print(f"Count: {count}")
    count += 1
```

In [ ]:
```
### Exercise 9: Factorial Calculator
# Write a function that calculates the factorial of a number using a loop
# Factorial of n = n * (n-1) * (n-2) * ... * 1

def factorial(n):
    # Your code here
    if n < 0:
        return "Error: Factorial not defined for negative numbers"

    result = 1
    for i in range(1, n + 1):
        result *= i

    return result

# Test your function
for num in range(6):
    print(f"Factorial of {num} = {factorial(num)}")
```

In [ ]:
```
### Exercise 10: Data Analysis
# You're given a list of dictionaries, each representing a student's score on diffe
# Calculate the average score for each student and identify the student with the hi

students_data = [
    {"name": "Alice", "scores": [88, 92, 95, 85]},
    {"name": "Bob", "scores": [90, 87, 88, 81]},
    {"name": "Charlie", "scores": [92, 95, 96, 93]},
    {"name": "David", "scores": [78, 85, 91, 89]}
]

def analyze_student_scores(students):
    results = {}
    highest_avg = 0
    top_student = None

    for student in students:
        name = student["name"]
        scores = student["scores"]
```

```
        avg_score = sum(scores) / len(scores)
        results[name] = avg_score

        if avg_score > highest_avg:
            highest_avg = avg_score
            top_student = name

    return {
        "individual_averages": results,
        "top_student": top_student,
        "top_average": highest_avg
    }

# Test your function
analysis = analyze_student_scores(students_data)
print("\nStudent Analysis:")
print("Individual Averages:")
for name, avg in analysis["individual_averages"].items():
    print(f"  {name}: {avg:.2f}")
print(f"Top Student: {analysis['top_student']} with average of {analysis['top_avera
```

In [ ]:

```
## Final Challenge: Sales Data Analysis

# You're given a list of dictionaries representing monthly sales data for a store.
# Each dictionary contains the month, category, and sales amount.

sales_data = [
    {"month": "Jan", "category": "Electronics", "amount": 1200},
    {"month": "Jan", "category": "Books", "amount": 850},
    {"month": "Jan", "category": "Clothing", "amount": 980},
    {"month": "Feb", "category": "Electronics", "amount": 1450},
    {"month": "Feb", "category": "Books", "amount": 920},
    {"month": "Feb", "category": "Clothing", "amount": 1050},
    {"month": "Mar", "category": "Electronics", "amount": 1600},
    {"month": "Mar", "category": "Books", "amount": 780},
    {"month": "Mar", "category": "Clothing", "amount": 1200}
]

def analyze_sales(sales):
    # Calculate total sales per month
    monthly_sales = {}
    for item in sales:
        month = item["month"]
        amount = item["amount"]
        if month in monthly_sales:
            monthly_sales[month] += amount
        else:
            monthly_sales[month] = amount

    # Calculate total sales per category
    category_sales = {}
    for item in sales:
        category = item["category"]
        amount = item["amount"]
        if category in category_sales:
            category_sales[category] += amount
```

```
        else:
            category_sales[category] = amount

    # Find best performing month and category
    best_month = max(monthly_sales, key=monthly_sales.get)
    best_category = max(category_sales, key=category_sales.get)

    # Calculate overall total sales
    total_sales = sum(item["amount"] for item in sales)

    return {
        "monthly_sales": monthly_sales,
        "category_sales": category_sales,
        "best_month": best_month,
        "best_category": best_category,
        "total_sales": total_sales
    }

# Test your analysis function
sales_analysis = analyze_sales(sales_data)
print("\nSales Analysis:")
print("Monthly Sales:")
for month, amount in sales_analysis["monthly_sales"].items():
    print(f"  {month}: ${amount}")
print("\nCategory Sales:")
for category, amount in sales_analysis["category_sales"].items():
    print(f"  {category}: ${amount}")
print(f"\nBest Month: {sales_analysis['best_month']} with ${sales_analysis['monthly
print(f"Best Category: {sales_analysis['best_category']} with ${sales_analysis['cat
print(f"Total Sales: ${sales_analysis['total_sales']}")
```

Congratulations on completing the Python Data Analytics Workshop!

Key concepts covered:

- Conditional statements (if-elif-else)
- Functions and parameters
- Lists and list operations
- Dictionaries and dictionary operations
- For and while loops
- Data analysis with Python

In [ ]: